

Middleware de protection des routes — CodeIgniter 4

Concept

Dans CI4, un **middleware** s'appelle un **Filter**. Il intercepte la requête *avant* ou *après* le contrôleur.

Étape 1 — Créer le filtre **AuthFilter**

app/Filters/AuthFilter.php

```
<?php
namespace App\Filters;

use CodeIgniter\HTTP\RequestInterface;
use CodeIgniter\HTTP\ResponseInterface;
use CodeIgniter\Filters\FilterInterface;

class AuthFilter implements FilterInterface
{
    public function before(RequestInterface $request, $arguments = null)
    {
        $session = session();

        // Si pas connecté → redirection login
        if (!$session->get('user')) {
            return redirect()->to('/login')->with('erreur', 'Connectez-vous
pour accéder à cette page');
        }
    }

    public function after(RequestInterface $request, ResponseInterface
$response, $arguments = null)
    {
        // Rien à faire après
    }
}
```

Étape 2 — Créer le filtre **RoleFilter**

app/Filters/RoleFilter.php

```
<?php
namespace App\Filters;

use CodeIgniter\HTTP\RequestInterface;
use CodeIgniter\HTTP\ResponseInterface;
```

```
use CodeIgniter\Filters\FilterInterface;

class RoleFilter implements FilterInterface
{
    public function before(RequestInterface $request, $arguments = null)
    {
        $session = session();
        $user     = $session->get('user');

        // $arguments contient le(s) rôle(s) autorisé(s)
        // ex: ['admin'] ou ['admin', 'bibliothecaire']
        if (!$user || !in_array($user['role'], $arguments ?? [])) {
            return redirect()->to('/')->with('erreur', 'Accès refusé :
droits insuffisants');
        }
    }

    public function after(RequestInterface $request, ResponseInterface
$response, $arguments = null)
    {
        // Rien à faire après
    }
}
```

Étape 3 — Déclarer les filtres ([app/Config/Filters.php](#))

```
<?php
namespace Config;

use App\Filters\AuthFilter;
use App\Filters\RoleFilter;
use CodeIgniter\Config\BaseConfig;

class Filters extends BaseConfig
{
    // Déclaration des alias
    public array $aliases = [
        'csrf'      => \CodeIgniter\Filters\CSRF::class,
        'auth'      => AuthFilter::class,
        'role'      => RoleFilter::class,
    ];

    // Filtres globaux (toutes les routes)
    public array $globals = [
        'before' => [
            'csrf',
        ],
    ];

    // Filtres par méthode HTTP (optionnel)
```

```
public array $methods = [];  
  
// Filtres par route spécifique (optionnel – voir étape 4)  
public array $filters = [];  
}
```

Étape 4 — Appliquer les filtres sur les routes

app/Config/Routes.php

```
// Route publique – aucun filtre  
$routes->get('/login', 'AuthController::form');  
$routes->post('/login', 'AuthController::login');  
$routes->get('/livres', 'LivreController::index');  
  
// Routes protégées – utilisateur connecté uniquement  
$routes->group('', ['filter' => 'auth'], function($routes) {  
    $routes->get('/livres/ajouter', 'LivreController::form');  
    $routes->post('/livres/store', 'LivreController::store');  
    $routes->post('/livres/preter/(:num)',  
    'EmpruntController::preter/$1');  
    $routes->post('/livres/retourner/(:num)',  
    'EmpruntController::retourner/$1');  
});  
  
// Routes réservées à l'admin  
$routes->group('admin', ['filter' => 'role:admin'], function($routes) {  
    $routes->get('dashboard',  
    'Admin\DashboardController::index');  
    $routes->post('livres/supprimer/(:num)',  
    'Admin\LivreController::supprimer/$1');  
    $routes->get('utilisateurs',  
    'Admin\UserController::index');  
});  
  
// Routes accessibles à admin ET bibliothécaire  
$routes->group('gestion', ['filter' => 'role:admin,bibliothecaire'],  
function($routes) {  
    $routes->get('emprunts',  
    'Gestion\EmpruntController::index');  
    $routes->get('emprunts/retards',  
    'Gestion\EmpruntController::retards');  
});
```

Étape 5 — Contrôleur d'authentification

app/Controllers/AuthController.php

```
<?php
namespace App\Controllers;

use App\Models\UserModel;

class AuthController extends BaseController
{
    public function form()
    {
        return view('auth/login');
    }

    public function login()
    {
        $model = new UserModel();

        $email    = $this->request->getPost('email');
        $password = $this->request->getPost('password');

        $user = $model->where('email', $email)->first();

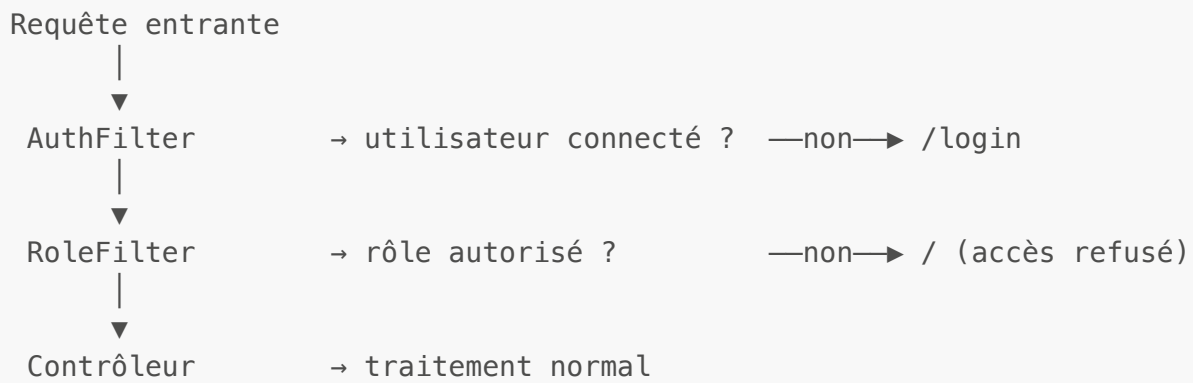
        if (!$user || !password_verify($password, $user['password'])) {
            return view('auth/login', [
                'erreur' => 'Email ou mot de passe incorrect'
            ]);
        }

        // Stocker uniquement les données non sensibles en session
        session()->set('user', [
            'id'    => $user['id'],
            'nom'   => $user['nom'],
            'email' => $user['email'],
            'role'  => $user['role'], // 'admin' | 'bibliothecaire' |
'lecteur'
        ]);

        return redirect()->to('/livres');
    }

    public function logout()
    {
        session()->destroy();
        return redirect()->to('/login');
    }
}
```

Récapitulatif du flux



Points clés à retenir

Élément	Rôle
<code>FilterInterface</code>	Contrat que tout filtre doit implémenter
<code>before()</code>	S'exécute avant le contrôleur
<code>\$arguments</code>	Paramètres passés dans la route (<code>role:admin</code>)
<code>session()->get('user')</code>	Source de vérité de l'identité
<code>redirect()->to()</code>	Sortie propre du filtre en cas de refus
Groupe de routes	Appliquer un filtre à plusieurs routes d'un coup